

Solar Paradox

66050014 กฤษฎา จารุกิตติคุณชัย

โครงการนี้เป็นส่วนหนึ่งของรายวิชาโครงการเชิงปฏิบัติ (05506239)

หลักสูตรปริญญาวิทยาศาสตรบัณฑิต วิทยาการคอมพิวเตอร์

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2568

1. ที่มาและความสำคัญของโครงการ

ในปัจจุบัน อุตสาหกรรมเกมแนวแก้ปริศนา (Puzzle Games) โดยเฉพาะกลุ่มเกมมุมมองจากด้านบน (Top-down) หรือแนวดันกล่อง (Sokoban) ได้รับความนิยมอย่างต่อเนื่อง เนื่องจากเป็นเกมที่ช่วยฝึกฝนทักษะการคิดวิเคราะห์ อย่างไรก็ตาม เกมปริศนาในตลาดส่วนใหญ่มักมีรูปแบบการเล่นที่ถูกจำกัดอยู่บนพื้นที่ 2 มิติแบบแบนราบตายตัว (Static 2D Grid) ผู้เล่นเพียงแค่ปฏิสัมพันธ์กับวัตถุในฉากระดับเดียว ทำให้ความท้าทายและการใช้จินตนาการเชิงพื้นที่ (Spatial Thinking) ถูกจำกัดลง โครงการนี้จึงเล็งเห็นโอกาสในการสร้างสรรค์นวัตกรรมใหม่ให้กับเกมแนว Puzzle ด้วยการขยายกรอบพื้นที่เดิมๆ

จากข้อจำกัดดังกล่าว จึงเป็นที่มาของการพัฒนาโครงการเกม **"Solar Paradox"** ซึ่งเป็นเกม 2D Top-down Puzzle ที่นำเสนอแนวคิดทางตรรกะแบบวนซ้ำ หรือ **"โลกซ้อนโลก" (Recursive & Sub-world Mechanics)** เป็นแกนหลัก (Core Gameplay) แทนที่กล่องเป้าหมายจะเป็นเพียงอุปสรรคกีดขวางธรรมดา แต่ภายในกล่องเหล่านั้นจะทำหน้าที่เป็น "ประตูมิติ (Gateway)" ที่เชื่อมด่านย่อย (Sub-world) เอาไว้ ผู้เล่นสามารถเคลื่อนที่ตัวละครนักบินอวกาศเข้าไปไขปริศนาในกล่อง และยังสามารถใช้กลไก **"ดันกล่องใบหนึ่ง เข้าไปในกล่องอีกใบหนึ่ง" (Box-in-Box)** เพื่อสร้างเส้นทางข้ามมิติ เหตุผลที่เลือกพัฒนาในหัวข้อนี้ เพื่อต้องการท้าทายขีดจำกัดในการเขียนโปรแกรมเพื่อจัดการสถานะของฉาก และนำเสนอรูปแบบการแก้ปัญหาเชิงซ้อนที่แปลกใหม่

กลุ่มเป้าหมายหลักของเกม Solar Paradox คือ กลุ่มผู้เล่นเกมคอมพิวเตอร์ตั้งแต่อายุ 10 ปีขึ้นไป ที่ชื่นชอบความท้าทายด้านตรรกะ (Logical Thinkers) และผู้ที่ชื่นชอบเกมแนวอินดี้ปริศนา (Indie Puzzle) ที่ต้องการประสบการณ์การเล่นที่ต้องใช้ทั้งจินตนาการและการวางแผนล่วงหน้า

คุณค่าหลักของโครงการนี้คือการมอบประสบการณ์ **"Eureka Moment"** หรือความรู้สึกตื่นเต้นเมื่อผู้เล่นสามารถไขความลับของมิติที่ซ้อนทับกันได้สำเร็จ เกมถูกออกแบบมาให้ผู้เล่นรู้สึกเหมือนกำลังสำรวจความลึกซึ้งของอวกาศ ภายใต้งานภาพสไตล์ 2D Pixel Art ที่เข้าถึงง่าย แต่แฝงไปด้วยความลึกซึ้งของตรรกะ ผู้เล่นจะได้ฝึกทักษะการวางแผน การจัดลำดับความคิด และความตระหนักรู้เชิงพื้นที่ (Spatial Awareness) ในระดับที่สูงขึ้นกว่าเกมปริศนาทั่วไป

สมมติฐานสำคัญของโครงการนี้คือ การประยุกต์ใช้เทคนิคเชิงโปรแกรมมิ่งร่วมกับการควบคุมมุมมอง (Virtual Camera) และ Raycast 2D จะสามารถจำลองระบบมิติซ้อนมิติได้อย่างไร้รอยต่อ (Seamless) โดยขอบเขตเบื้องต้นคือการสร้างระบบต้นแบบ (Prototype) ที่สามารถเล่นได้จริงบนแพลตฟอร์ม PC (Windows) และมีระบบหลักในการเคลื่อนที่ ควบคุม และปฏิสัมพันธ์ระหว่างโลกหลักและโลกย่อยได้อย่างเสถียรและไร้ข้อผิดพลาด (Bug-free) ในระดับกลไกพื้นฐาน

2. งานและเทคโนโลยีที่เกี่ยวข้อง

ในการพัฒนาโครงการเกม "Solar Paradox" ผู้จัดทำได้ศึกษาและรวบรวมข้อมูลจากผลงานเกมที่เกี่ยวข้อง รวมถึงเทคโนโลยีและเครื่องมือต่างๆ เพื่อนำมาใช้อ้างอิงและเป็นแนวทางในการพัฒนาระบบกลไกของเกม โดยมีรายละเอียดดังนี้

2.1 เกมอ้างอิงและกลไกเกมที่เกี่ยวข้อง (Reference Games and Mechanics)

แรงบันดาลใจหลักในการออกแบบโครงการคือเกม Patrick's Paradox ซึ่งได้ต่อยอดกลไกจากเกมปริศนาดันกล่องคลาสสิกอย่าง Sokoban โดยหลายข้อจำกัดด้านพื้นที่ 2 มิติตายตัว สู่การนำเสนอกลไกการวนซ้ำเชิงพื้นที่ (Spatially Recursive) ผ่านระบบกล่องภายในกล่อง (Box-within-a-Box) ที่อนุญาตให้ผู้เล่นผลักและมุดเข้าไปไขปริศนาในมิติย่อย (Sub-world) ได้ โครงการ Solar Paradox ได้นำความซับซ้อนเชิงมิตินี้มาประยุกต์เป็นแกนหลักของการเล่น (Core Gameplay) ภายใต้ธีมการสำรวจอวกาศ เพื่อท้าทายให้ผู้เล่นต้องใช้ทั้งความคิดเชิงตรรกะและจินตนาการเชิงพื้นที่ในการแก้ปัญหาอย่างลึกซึ้ง

2.2 เทคโนโลยีและเอนจินที่เกี่ยวข้อง (Related Technologies and Engines)

ผู้จัดทำเลือกใช้ Unity Engine (เวอร์ชัน 6) เป็นเครื่องมือหลัก เนื่องจากมีระบบ 2D Tilemap ที่ช่วยสร้างแผนที่ตาราง (Grid-based) ได้อย่างแม่นยำ ผสานกับ Physics 2D และ Raycast 2D สำหรับตรวจจับลอจิกการชนล่วงหน้า (Look-ahead Collision) เพื่อป้องกันการเคลื่อนที่ที่ทับซ้อนก่อนเกิดการข้ามมิติ ในด้านการควบคุมตรรกะ มีการใช้ภาษา C# และการเขียนโปรแกรมเชิงวัตถุ (OOP) เพื่อจัดการสถานะและพิกัดของกล่องแต่ละใบให้แยกขาดจากกัน (Encapsulation) ช่วยให้ระบบคำนวณการสลับมิติได้โดยข้อมูลไม่ขัดแย้งกัน นอกจากนี้ยังประยุกต์ใช้กล้องเสมือน (Virtual Camera) สร้างภาพลวงตาของการชุมสลับมิติให้ลื่นไหลไร้รอยต่อ

2.3 สรุปเหตุผลผลการเลือกใช้แนวทาง (Conclusion of Approach Selection)

จากการศึกษางานข้างต้น ผู้จัดทำจึงตัดสินใจเลือกกลไกปริศนาเชิงซ้อนจาก Patrick's Paradox เข้ากับประสิทธิภาพของ Unity Engine ที่มีความยืดหยุ่นสูงในการจัดการพื้นที่และการเรนเดอร์ภาพ ช่วยลดข้อจำกัดในการสร้างระบบฟิสิกส์พื้นฐาน ทำให้ผู้จัดทำสามารถมุ่งเน้นไปที่การออกแบบสถาปัตยกรรมโค้ด (Code Architecture) ด้วยภาษา C# เพื่อจัดการลอจิกโลกซ้อนโลกที่ซับซ้อนได้อย่างมีประสิทธิภาพ และถ่ายทอดประสบการณ์การเล่นออกมาได้ตรงตามเป้าหมายอย่างสมบูรณ์

3. ขอบเขต เป้าหมาย และข้อกำหนด (Scope, Goals, and Requirements)

โครงการเกม "Solar Paradox" มีการกำหนดเป้าหมายและขอบเขตการพัฒนา เพื่อให้ระบบกลไกปริศนา "โลกซ้อนโลก" สามารถทำงานได้อย่างสมบูรณ์ภายใต้ระยะเวลาและทรัพยากรที่จำกัด โดยมีรายละเอียดดังนี้

3.1 เป้าหมายของโครงการและเป้าหมายของเกม (Project and Game Goals)

เป้าหมายหลักของโครงการคือการพัฒนาตัวต้นแบบเกม (Playable Prototype) แนว 2D Top-down Puzzle ที่สามารถสาธิตระบบมิติซ้อนมิติ (Recursive Dimension) ได้อย่างเสถียรและไร้ข้อผิดพลาด (Bug-free) ในส่วนเป้าหมายของเกม ผู้เล่นจะได้รับบทเป็นนักบินอวกาศที่ต้องไขปริศนาในแต่ละด่าน โดยมีภารกิจคือการผลักวัตถุหรือนำตัวละครไปวางบน "จุดเป้าหมาย (Goal Plate)" ให้ครบตามเงื่อนไข เพื่อเปิดประตูมิติสู่ด่านถัดไป

3.2 Gameplay หลักและวงจรการเล่น (Core Gameplay and Core Loop)

ผู้เล่นจะควบคุมตัวละครให้เดินและดันวัตถุไปตามช่องตาราง (Grid-based) โดยมีความท้าทายหลักคือการไขปริศนาข้ามมิติ วงจรการเล่นจะเริ่มต้นจากการที่ผู้เล่นต้องวิเคราะห์พื้นที่และตำแหน่งเป้าหมาย ทั้งในมุมมองของโลกหลักและพื้นที่มิติย่อยที่ซ่อนอยู่ภายในกล่อง จากนั้นผู้เล่นจะต้องลงมือแก้ปัญหาผ่านกลไกพิเศษของเกม เช่น "การมุดข้ามมิติ" (Dimension Hopping) เพื่อเข้าไปจัดการพื้นที่ในกล่อง หรือ "การดันกล่องซ้อนกล่อง" (Box-in-Box) เพื่อเคลื่อนย้ายสิ่งกีดขวางข้ามมิติ เมื่อผู้เล่นสามารถจัดเรียงพื้นที่และนำวัตถุหรือตัวละครไปวางทับจุดเป้าหมาย (Goal) ได้ครบตามเงื่อนไข ประตูมิติทางออกจะเปิดขึ้น เพื่อนำผู้เล่นเดินทางเข้าสู่ด่านถัดไปที่มีความซับซ้อนยิ่งขึ้นอย่างลื่นไหลไร้รอยต่อ (Seamless Transition)

3.3 สิ่งที่ยอยู่นอกเหนือขอบเขต (Out of Scope)

เพื่อควบคุมระยะเวลาการพัฒนา โครงการนี้จะไม่มีระบบผู้เล่นหลายคน (Multiplayer) ไม่มีศัตรู AI ที่เคลื่อนที่โจมตีได้ และไม่มีการสุ่มด่าน (Procedural Generation) โดยด่านทั้งหมดจะถูกออกแบบไว้ล่วงหน้า (Hand-crafted Levels) เพื่อควบคุมความสมดุลของปริศนาอย่างรอบคอบ

3.4 ข้อกำหนดและข้อจำกัด (Requirements and Constraints)

ตัวเกมถูกพัฒนาสำหรับเล่นบนระบบปฏิบัติการ Windows (PC) ผ่านการควบคุมด้วยคีย์บอร์ดเป็นหลัก สำหรับข้อจำกัดด้านทรัพยากร ผู้จัดทำจะมุ่งเน้นความสำคัญไปที่การพัฒนาระบบโค้ดและตรรกะ (Logic) เป็นหลัก จึงเลือกใช้งานกราฟิก 2D Pixel Art จากแพลตฟอร์ม itch.io ภายใต้สิทธิ์การใช้งานที่ถูกต้อง

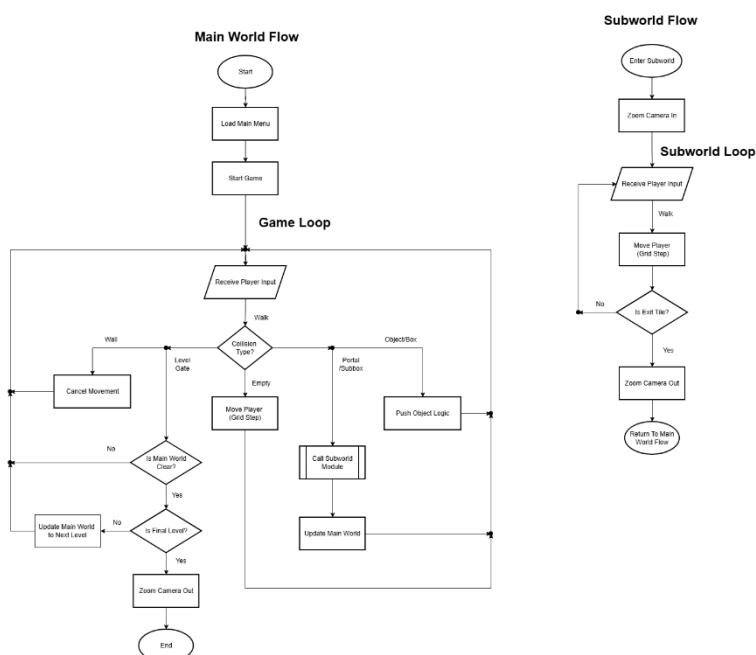
4. การออกแบบหรือแนวทางเชิงเทคนิค

ในการพัฒนาเกม "Solar Paradox" ผู้จัดทำได้ออกแบบโครงสร้างของระบบและสถาปัตยกรรมซอฟต์แวร์ให้มีความยืดหยุ่น เพื่อรองรับความซับซ้อนของกลไกการเล่นแบบมิติซ้อนมิติ โดยมีการออกแบบในแต่ละส่วนดังนี้

4.1 กระแสการเล่นเกมรวม (Gameplay Flow)

กระแสการทำงานหลักของเกมถูกแบ่งออกเป็น 2 ลูป (Loop) ที่ทำงานสอดประสานกัน ดังนี้:

- **Main World Flow:** เป็นลูปหลักที่เริ่มต้นจากเมนูเข้าสู่ตัวเกม เมื่อผู้เล่นสั่งเดิน ระบบจะตรวจสอบการชน (Collision) หากผู้เล่นเดินชน "กล่องมิติ (Portal/Subbox)" ระบบจะเรียกใช้โมดูลมิตีย่อย (Call Subworld Module) ทันที
- **Subworld Flow:** เมื่อโมดูลมิตีย่อยทำงาน กล่องจะซุมเข้าสู่ภายในกล่องและเข้าสู่ลูปลงการเล่นย่อย ผู้เล่นจะไขปริศนาอยู่ภายในนี้จนกว่าจะเดินไปที่ "ช่องทางออก (Exit Tile)" ระบบจึงจะซุมกล่องออก และส่งผู้เล่นกลับสู่ Main World Flow
- **เงื่อนไขการผ่านด่าน:** ใน Main World จะมีการตรวจสอบความสำเร็จเสมอ หากเคลียร์ด่านได้ (Main World Clear) ระบบจะโหลดด่านถัดไปหรือจบเกม (End) หากเป็นด่านสุดท้าย



4.2 การจัดการสถานะของเกม (Game State Management)

เพื่อป้องกันข้อผิดพลาดและบั๊กจากการรับคำสั่งซ้อนทับกัน ผู้จัดทำได้ควบคุมลอจิกของเกมผ่าน GameManager โดยแบ่งสถานะออกเป็น 5 สถานะหลัก ได้แก่:

- 1 **Init & Menu State:** สถานะเริ่มต้นของระบบเพื่อโหลดทรัพยากรและแสดงผลหน้าต่าง UI เมนูหลัก
- 2 **Playing State:** สถานะหลักที่ระบบเปิดรับอินพุต ผู้เล่นสามารถควบคุมตัวละคร ดันกล่อง และใช้งานตรรกะมิติซ้อนมิติได้เต็มรูปแบบ
- 3 **Paused State:** สถานะหยุดพักเมื่อผู้เล่นกดเปิดเมนูระหว่างเล่น ระบบจะหยุดการทำงานของฟิสิกส์และลอจิกเกมชั่วคราว
- 4 **Transition State (รอยต่อการสลับมิติ):** สถานะพิเศษที่จะทำงานเมื่อกำลังกำลังซูมเข้า-ออกมิตีย่อย หรือกำลังโหลดด่านใหม่ ในสถานะนี้ระบบจะทำการ "บล็อกอินพุต (Block Input)" เพื่อป้องกันไม่ให้ผู้เล่นขยับตัวจนเกิดบั๊กทะลุกำแพง
- 5 **Level Complete State:** เมื่อบรรลุเงื่อนไขด่าน ระบบจะล็อกการควบคุม เล่นแอนิเมชันผ่านด่าน และเตรียมข้อมูลด่านถัดไป

4.3 โครงสร้างฉากและส่วนติดต่อผู้ใช้ (Scene Structure and UI Flow)

- **Scene Structure:** ใช้สถาปัตยกรรมแบบแยกฉาก (Multi-Scene) โดยมี Core Scene ที่เปิดค้างไว้ตลอดเพื่อบรรจุระบบจัดการหลัก และ Level Scenes ที่ใช้เก็บข้อมูลแผนที่และปริศนาของด่านนั้นๆ ซึ่งจะถูกโหลดหรือลบออก (Load/Unload) เมื่อมีการเปลี่ยนด่าน
- **UI Flow:** ออกแบบให้เรียบง่าย ไม่บดบังทัศนวิสัย โดยมีหน้า Main Menu (เริ่มเกม, เลือกด่าน), หน้า Pause Menu (ทำต่อ, เริ่มด่านใหม่, กลับเมนูหลัก) และส่วน HUD ที่แสดงข้อมูลด่านปัจจุบัน พร้อมปุ่มลัด (Shortcut) สำหรับกดเริ่มด่านใหม่ทันทีหากผู้เล่นติดขัด

4.4 ความสัมพันธ์ของวัตถุภายในเกม (Entity Relationship)

ตัวเกมใช้หลักการเขียนโปรแกรมเชิงวัตถุ เพื่อรองรับระบบกล่องซ้อนกล่องอย่างเป็นระบบ:

- **LevelManager & GridManager:** จัดการภาพรวมของด่านและบันทึกพิกัด (Coordinates) ของวัตถุบนตาราง
- **Box Entity:** วัตถุ "กล่องมิติ" ไม่ได้เป็นแค่สิ่งกีดขวาง แต่ภายในคลาสของมันจะมี GridManager ย่อยเป็นของตัวเอง ทำให้กล่อง 1 ใบทำหน้าที่เป็น "โลกจำลอง" ที่มีพิกัดภายในแยกขาดจากโลกหลักได้

4.5 กลไกการสลับพิกัด (Coordinate Mapping Mechanics)

หัวใจสำคัญของการข้ามมิติคือ "การสลับการอ้างอิงตาราง" เมื่อผู้เล่นเดินทะลุเข้าสู่กล่องมิติ ข้อมูลพิกัดของตัวละครจะถูกถอดจาก GridManager ของโลกหลักและส่งไปให้ตารางย่อยของกล่องใบนั้นดูแลแทน การจัดการแบบนี้ทำให้ระบบสามารถคำนวณการชนและการผลักวัตถุในแต่ละมิติได้อย่างอิสระและไม่ทับซ้อนกัน

5. เครื่องมือ เทคโนโลยี และทรัพยากรที่ใช้

ในการพัฒนาโครงการเกม "Solar Paradox" ผู้จัดทำได้เลือกใช้เครื่องมือ ซอฟต์แวร์ และ ทรัพยากรต่างๆ ที่เหมาะสมกับการสร้างเกม 2D และสอดคล้องกับข้อจำกัดด้านเวลาของโครงการ โดยมีรายละเอียดดังนี้

5.1 ซอฟต์แวร์และเครื่องมือพัฒนา (Software and Development Tools)

- **Game Engine:** Unity Engine (เวอร์ชัน 6) ใช้เป็นเครื่องมือหลักในการสร้างแผนที่ตาราง (Tilemap) จัดการระบบการชน (Physics 2D / Raycast 2D) และควบคุมการทำงานของกล้อง (Virtual Camera)
- **โปรแกรมเขียนโค้ด (IDE):** Visual Studio Code ใช้สำหรับการเขียนและแก้ไขสคริปต์ ควบคุมตรรกะเกมด้วยภาษา C#
- **ระบบจัดการเวอร์ชัน (Version Control):** Git และ GitHub ใช้สำหรับการจัดเก็บซอร์สโค้ด (Source Code) จัดการเวอร์ชันของการพัฒนา และสำรองข้อมูลโครงการเพื่อป้องกันความเสียหาย

5.2 ทรัพยากรและสินทรัพย์ภายในเกม (Game Assets and Resources)

เพื่อให้ผู้จัดทำสามารถมุ่งเน้นเวลาไปกับการเขียนสถาปัตยกรรมโค้ดและการออกแบบตรรกะมิติซ้อนโลก (Recursive Logic) ได้อย่างเต็มประสิทธิภาพ จึงมีการบริหารจัดการทรัพยากรดังนี้:

- **กราฟิกและสภาพแวดล้อม (Graphics & Environment):** โครงการนี้เลือกใช้ทรัพยากรภาพ 2D Pixel Art สำเร็จรูปจากแพลตฟอร์ม itch.io โดยคัดเลือกเฉพาะชุดกราฟิกที่มีสิทธิ์การใช้งานที่ถูกต้อง (เช่น สัญญาอนุญาต CC0 หรือตามเงื่อนไขอนุญาตของผู้สร้าง) และนำมาจัดองค์ประกอบใหม่ให้เข้ากับธีมอวกาศ
- **รูปแบบตัวอักษร (Fonts):** ใช้ฟอนต์ที่อ่านง่าย เข้ากับสไตล์ภาพ Pixel และมีสิทธิ์การใช้งานแบบเปิดเสรี (Free for commercial use)

5.3 ฮาร์ดแวร์สำหรับการพัฒนาและทดสอบ (Hardware Requirements)

การพัฒนาและการทดสอบโครงการทั้งหมดดำเนินการบนคอมพิวเตอร์ส่วนบุคคล (PC) ระบบปฏิบัติการ Windows โดยมีคุณสมบัติขั้นต่ำที่สามารถประมวลผล Unity Engine และเรนเดอร์ภาพ 2D ได้อย่างเสถียร และใช้แป้นพิมพ์ (Keyboard) เป็นอุปกรณ์อินพุตหลักในการทดสอบระบบการควบคุม

6. การทดสอบหรือการประเมินผล

ในการพัฒนาโครงการเกม "Solar Paradox" ผู้จัดทำได้กำหนดวิธีการทดสอบและประเมินผล เพื่อให้ตัวเกมมีประสิทธิภาพ ไร้ข้อผิดพลาด และมอบประสบการณ์การเล่นที่ดี โดยอ้างอิงวิธีการทดสอบสำหรับโครงการประเภทเกม ดังนี้

6.1 วิธีการทดสอบและเกณฑ์การประเมิน (Testing Methodology and Criteria)

ผู้จัดทำเลือกใช้วิธีการทดสอบ 2 ส่วนหลัก ได้แก่:

1. **Functional Testing (การทดสอบระบบลอจิกและบั๊ก):** ทดสอบโดยผู้พัฒนา เพื่อหาข้อผิดพลาดทางการโต้ตอบ (Interaction Issues) และบั๊ก (Bugs) ในระบบฟิสิกส์ การเคลื่อนที่ และการสลับมิติ
 2. **Playtesting (การทดสอบโดยผู้เล่นจริง):** เชิญกลุ่มตัวอย่างที่เป็นผู้เล่นทั่วไปจำนวนหนึ่งมาทดลองเล่น เพื่อประเมิน "ความสนุก (Fun Factor)" "ประสบการณ์ผู้เล่น (Player Experience)" และ "ความสมดุลของเกม (Game Balance)"
- **เกณฑ์การประเมิน (Metrics):** วัดผลจากอัตราการผ่านด่าน (Completion Rate), จำนวนครั้งที่ต้องกดเริ่มต้นใหม่ (Reset Count), ระยะเวลาที่ใช้ในการแก้ปริศนาต่อด่าน

6.2 ตัวอย่างกรณีทดสอบ (Test Cases สำหรับ Interaction & Bugs)

เพื่อรับประกันความเสถียรของระบบมิติซ้อนมิติ ได้มีการกำหนด Test Cases ที่สำคัญ เช่น:

- **กรณีทดสอบที่ 1:** ดันกล่องธรรมดาชนกับกำแพง (ผลที่คาดหวัง: กล่องต้องหยุดและไม่ทะลุกำแพง)
- **กรณีทดสอบที่ 2:** ตัวละครเดินเข้าสู่กล่องมิติ (ผลที่คาดหวัง: กล่องต้องซูมเข้าสู่ตารางย่อยอย่างลื่นไหล และระบบการควบคุมต้องสลับไปคำนวณที่ตารางย่อยทันที)
- **กรณีทดสอบที่ 3:** ดันกล่องมิติใบที่ 1 เข้าไปซ้อนในกล่องมิติใบที่ 2 (Box-in-Box) (ผลที่คาดหวัง: กล่องใบที่ 1 ต้องเข้าไปอยู่ในพิกัดย่อยของกล่องใบที่ 2 โดยที่ข้อมูลตารางภายในของกล่องใบที่ 1 ต้องไม่สูญหาย)

6.3 ผลการทดสอบเบื้องต้น สิ่งที่พบ และแนวทางปรับปรุง (Findings and Refinements)

จากการนำตัวเกมนั้นแบบไปผ่านกระบวนการ Playtesting และ Functional Testing พบประเด็นต่างๆ ดังนี้:

- **ปัญหาด้านบั๊กและการโต้ตอบ (Bug / Interaction Issue):** ในช่วงแรกพบว่าหากผู้เล่นดันวัตถุพร้อมกับการกดปุ่มทิศทางรัวๆ ตัวละครอาจเดินทับซ้อนกับวัตถุได้ แนวทางปรับปรุง: ได้ทำการแก้ไขโค้ดในระบบ Raycast ให้ตรวจสอบสถานะการชนให้เสร็จสิ้นก่อนรับคำสั่งเดินในก้าวถัดไป ทำให้ปัญหาการทับซ้อนหมดไป
- **ความสมดุลของเกม (Game Balance):** ผล Playtesting ชี้ให้เห็นว่า ระดับความยาก (Difficulty Curve) ก้าวกระโดดเร็วเกินไปในด่านที่ 3 ทำให้ผู้เล่นใช้เวลานานและเกิดความหงุดหงิด แนวทางปรับปรุง: ทำการปรับสมดุลใหม่ โดยเพิ่ม "ด่านเชื่อมต่อ" ที่ค่อยๆ สอนกลไกทีละอย่าง (เช่น สอนการดันกล่องเข้ามิติ ก่อนจะไปเจอการดันกล่องซ้อนกล่อง) เพื่อให้ผู้เล่นเรียนรู้ลอจิกอย่างเป็นขั้นตอน
- **ประสบการณ์ผู้เล่น (Player Experience):** ผู้เล่นส่วนใหญ่ชื่นชอบความแปลกใหม่ของระบบโลกซ้อนโลก และรู้สึกพึงพอใจ (Satisfied) เมื่อไขปริศนาที่ซับซ้อนได้สำเร็จ ถือว่าตัวเกมตอบโจทย์ด้านความสนุกและการใช้ความคิดเชิงตรรกะได้ดีเยี่ยม

7. ข้อจำกัด ประเด็นจริยธรรม และแนวทางพัฒนาต่อ

ในการพัฒนาโครงการเกม "Solar Paradox" ผู้จัดทำได้กำหนดวิธีการทดสอบและประเมินผล เพื่อให้ตัวเกมมีประสิทธิภาพ ไร้ข้อผิดพลาด และมอบประสบการณ์การเล่นที่ดี โดยอ้างอิงวิธีการทดสอบสำหรับโครงการประเภทเกม ดังนี้

7.1 ข้อจำกัดและอุปสรรคในการพัฒนา (Limitations and Obstacles)

- **ข้อจำกัดด้านเวลาและทรัพยากร:** เนื่องจากโครงการมีระยะเวลาจำกัด ทำให้ไม่สามารถพัฒนาระบบ AI ของศัตรู หรือสร้างอนิเมชันที่ซับซ้อนได้ ตัวเกมจึงต้องจำกัดขอบเขต (Scope) ไว้ที่การแก้ปริศนาเชิงพื้นที่ (Spatial Puzzle) เป็นหลัก
- **ความซับซ้อนของสถาปัตยกรรมข้อมูล:** การออกแบบระบบพิกัดที่สามารถซ้อนทับกันได้แบบอนันต์ (Infinite Recursion) มีความเสี่ยงที่จะทำให้หน่วยความจำเต็ม (Memory Leak) ผู้จัดทำจึงต้องตั้งค่าขอบเขตจำกัดความลึกของการซ้อนมิติไว้เพื่อรักษาความเสถียรของระบบ

7.2 ประเด็นด้านจริยธรรม กฎหมาย และความเหมาะสม

- **ทรัพย์สินทางปัญญาและลิขสิทธิ์ (Copyrighted Assets):** งานกราฟิกทั้งหมดที่ไม่ได้สร้างขึ้นเอง ถูกนำมาจากแหล่งแจกจ่ายทรัพยากร (itch.io) โดยผู้จัดทำได้ตรวจสอบเงื่อนไขการใช้งาน (License) อย่างเคร่งครัด โดยเลือกใช้เฉพาะ Asset ที่อนุญาตให้ใช้ฟรี (เช่น CC0)

- **ความเหมาะสมตามวัย (Age Appropriateness):** เกม "Solar Paradox" เป็นเกมแนวแก้ปริศนาที่ไม่มีภาพความรุนแรง ไม่มีเนื้อหาเรื่องเพศ และไม่มีการใช้ภาษาที่ไม่เหมาะสม ตัวเกมจึงจัดอยู่ในเรตที่เหมาะสมสำหรับทุกเพศทุกวัย (E for Everyone) ช่วยส่งเสริมทักษะการคิดวิเคราะห์เชิงตรรกะได้เป็นอย่างดี
- **ความเสี่ยงด้านประสบการณ์ผู้เล่น (Player Experience Risk):** แม้จะไม่มีภาพความรุนแรง แต่ความซับซ้อนของปริศนาอาจทำให้ผู้เล่นบางกลุ่มเกิดความเครียดหรือหงุดหงิด (Frustration) จากสถานะคิดไม่ออก (Cognitive Overload) หรือการเดินผิดพลาดจนเกมติดขัด (Soft-lock) ผู้จัดทำจึงได้ออกแบบปุ่ม "Restart (เริ่มต้นใหม่ทันที)" ไว้เพื่อให้ผู้เล่นสามารถเริ่มต้นใหม่ได้ตลอดเวลาโดยไม่รู้สึกลังเล

7.3 แนวทางพัฒนาต่อ (Future Development)

- **ระบบสร้างด่านของผู้เล่น (Level Editor / User-Generated Content):** พัฒนาระบบให้ผู้เล่นสามารถสร้างด่านปริศนามิตีซ้อนมิติของตนเอง และอัปโหลดขึ้นสู่เซิร์ฟเวอร์เพื่อท้าทายผู้เล่นคนอื่นๆ ในคอมมูนิตี
- **การพัฒนาศิลปะและเนื้อเรื่อง (Original Art and Narrative):** สร้างงานกราฟิก (Original Art Assets) ที่เป็นเอกลักษณ์ของตนเองทั้งหมด และเพิ่มระบบเนื้อเรื่อง (Storyline) ที่เล่าถึงประวัติของนักบินอวกาศและความลับของมิติซ้อนมิติ เพื่อเพิ่มแรงจูงใจในการเล่นให้มากขึ้น
- **การขยายแพลตฟอร์ม (Cross-Platform):** พัฒนาระบบการควบคุมให้รองรับ Controller และหน้าจอสัมผัส (Touch Screen) เพื่อเตรียมพอร์ตตัวเกมลงสู่แพลตฟอร์มคอนโซลและสมาร์ทโฟนในอนาคต

8. เอกสารอ้างอิง

ประภาพร เกียรติเอกสิทธิ์. 2565. คู่มือสร้างเกม 2D ด้วย Unity. กรุงเทพฯ. โปรวิชั่น.

Microsoft. 2024. C# documentation. [Online].

<https://learn.microsoft.com/en-us/dotnet/csharp/>.

Traylor, P. 2022. Patrick's Parabox. [Online]. <https://www.patricksparabox.com>.

Unity Technologies. 2024. Unity User Manual. [Online]. <https://docs.unity3d.com>.

ในการจัดทำโครงการฉบับนี้ ผู้จัดทำได้ใช้เทคโนโลยีปัญญาประดิษฐ์ (AI) เป็นเครื่องมือช่วยในการค้นคว้าหาแนวทางแก้ปัญหาทางเทคนิคระหว่างเขียนโปรแกรม และช่วยตรวจทานข้อผิดพลาดในเอกสาร โดยผู้จัดทำเป็นผู้ลงมือปฏิบัติ เขียนระบบหลัก และตัดสินใจนำคำแนะนำมาปรับใช้จริง ซึ่งมีรายละเอียดดังนี้:

- โมเดล AI ที่ใช้งาน (AI Model Used): Google Gemini 3.1 Pro
- ตัวอย่างชุดคำสั่งที่ใช้ (Prompts Used):

ส่วนที่ 1: ด้านการเขียนโปรแกรมและแก้บั๊ก (Game Development & Debugging)

- วิธีเขียน C# ใน Unity ให้ตัวละครเดินแบบ Grid-based ที่ละช่อง แล้วไม่เดินทะลุกำแพงใหม่? ควรใช้ Raycast2D เช็กการชนแบบไหนถึงจะแม่นยำที่สุด?"
- กำลังทำระบบกล่องซ้อนกล่อง (Box-in-Box) ตอนนี้ติดปัญหาเรื่องพิกัดซ้อนทับกัน พอจะมีวิธีออกแบบคลาส (Class) เพื่อแยกการจัดการ Grid ของโลกหลักกับโลกในกล่องออกจากกันไหม?"
- อยากทำระบบซูมกล้องเข้า-ออกตอนที่ตัวละครเดินเข้าประตูมิติ ใช้ Cinemachine ทำยังไงให้ภาพดูสมูท และควรเขียนสคริปต์ล็อกไม่ให้ผู้เล่นกดปุ่มเดินตอนกล้องกำลังซูม ยังไงดี ป้องกันบั๊กเดินทะลุ"

ส่วนที่ 2: ด้านการเรียบเรียงเอกสาร (Document Refinement)

- เรากำลังทำรายงานเล่มจบของเกม Solar Paradox ช่วยอธิบายระบบมิติซ้อนมิติที่เราทำให้ออกมาเป็นภาษาเอกสารทางเทคนิคหน่อย
- ช่วยเรียบเรียงเรื่อง Game State ของเกม Solar Paradox ให้ดูเป็นทางการและอ่านง่ายขึ้นหน่อย